



**Centro Universitário UniAcademia**

**INTELIGÊNCIA ARTIFICIAL E MACHINE LEARNING**

**Trabalho Final de IA e ML**

**IGOR GABRIEL RODRIGUES  
PEDRO JOSÉ GUIMARÃES COELHO  
RODRIGO DEBORTOLI DE SOUZA  
DAVI DOS SANTOS MORAIS  
LUIZ FELIPE LAMAS MISHIMA**

**JUIZ DE FORA**

**2025**

# 1. INTRODUÇÃO

Nas últimas décadas, a Interação Humano-Computador (IHC) passou por uma transformação radical, impulsionada diretamente pela evolução da Inteligência Artificial (IA) e do Aprendizado de Máquina (*Machine Learning* - ML). Dentro desse cenário, os chatbots deixaram de ser meros agregadores de scripts estáticos para assumir o papel de agentes dinâmicos na linha de frente do suporte e da automação. A principal missão da IHC é tornar a tecnologia mais acessível e intuitiva, e os chatbots cumprem esse papel ao adotar a linguagem natural — a interface mais antiga e eficiente da humanidade. O objetivo deste trabalho consiste em descrever a importância dessas ferramentas e demonstrar como a integração de IA e ML otimiza a comunicação humano-máquina, com foco especial no desenvolvimento de uma arquitetura híbrida que combina a precisão de uma Base de Conhecimento com a flexibilidade dos Grandes Modelos de Linguagem (LLMs).

Para compreender o estado da arte dessas soluções, é fundamental observar sua evolução histórica. A primeira fase, compreendida entre os anos 1960 e 2000, foi a Era dos Sistemas Baseados em Regras. Modelos pioneiros como a *ELIZA* (1966), criada no MIT, e a *ALICE* (1995), utilizavam linguagens de marcação como a AIML e substituição de padrões (*pattern matching*) para simular conversas. Apesar de inovadores, eram sistemas deterministas que não compreendiam o texto, quebrando ao menor sinal de erro de digitação. Com a virada do milênio e o aumento da capacidade computacional, a década de 2010 inaugurou a era do Machine Learning aplicado ao Processamento de Linguagem Natural (PLN). Surgiram assistentes comerciais como *Siri* e *Alexa*, que substituíram as regras rígidas por modelos estatísticos capazes de realizar a classificação de intenções e a extração de entidades.

O verdadeiro salto tecnológico, contudo, ocorreu em 2017 com o lançamento do artigo "*Attention Is All You Need*" pelo Google, que introduziu a arquitetura *Transformer*. Essa tecnologia permitiu que as máquinas processassem palavras compreendendo o contexto global de uma frase de forma inédita. A partir daí, surgiram os LLMs, capazes de manter o histórico da conversa, entender nuances e gerar textos de forma fluida. Inicialmente, o acesso a esses modelos era restrito a grandes corporações (APIs fechadas), mas o movimento *open-source*, liderado pela plataforma Hugging Face, democratizou o setor. Funcionando como o "GitHub da IA", o Hugging Face disponibilizou milhares de modelos abertos, como *Llama* e *Mistral*, permitindo que desenvolvedores implementassem e realizassem o refinamento (*fine-tuning*) de IAs em seus próprios ecossistemas a custos drasticamente reduzidos.

Nesse contexto, justifica-se a relevância de estudar e desenvolver chatbots híbridos. A adoção de uma Base de Conhecimento atua como o alicerce de controle

e confiabilidade, funcionando como a "única fonte de verdade" (*Single Source of Truth*). Ela garante que as respostas sejam factuais e atualizadas, mitigando riscos de alucinações. O LLM, por sua vez, entra como um mecanismo complementar indispensável: ele lê os dados brutos recuperados da base, corrige erros de digitação da entrada do usuário, extrai a informação relevante e redige uma resposta natural e empática. Por fim, a escolha do modelo utilizado torna-se uma decisão arquitetural crítica, exigindo do desenvolvedor o equilíbrio entre custo, infraestrutura e privacidade, optando-se por modelos *open-source* (como os via Hugging Face) sempre que houver necessidade de independência tecnológica e customização profunda.

## 2. Fundamentação Teórica

Chatbots são sistemas de software projetados para interagir com seres humanos por meio de linguagem natural, seja por texto ou por voz, operando de forma contínua para automatizar tarefas e responder a dúvidas. O funcionamento dessas ferramentas depende da tecnologia adotada em seu núcleo de processamento, segmentando-se historicamente em três abordagens arquiteturais principais. A mais simples é a Arquitetura Baseada em Regras (*Rule-Based*), onde o comportamento é mapeado por árvores de decisão lógicas (if/else), resultando em respostas estáticas e limitadas a palavras-chave exatas. A segunda abordagem é a Recuperação de Informação (*Retrieval-Based*), que utiliza aprendizado estatístico para classificar a intenção do usuário e buscar, em um banco de dados, a resposta com maior pontuação de confiança. Por fim, o estado da arte é representado pela Geração por LLM (*Generative LLM*), baseada em redes neurais profundas que não buscam respostas prontas, mas geram textos palavra por palavra em tempo real, compreendendo semântica, contexto profundo e múltiplos idiomas de forma fluida.

O motor que viabiliza essa evolução é a Inteligência Artificial (IA), um campo macro da ciência da computação focado em criar máquinas capazes de executar tarefas de intelecto humano. Dentro da IA, os chatbots utilizam diversos algoritmos especializados. Na era focada em intenções, destacavam-se os algoritmos de Classificação e Extração, como o *Naive Bayes* e as *Support Vector Machines* (SVM), além dos *Conditional Random Fields* (CRF) para o reconhecimento de entidades nomeadas. Para a busca semântica e recuperação de informações em arquiteturas estruturadas, algoritmos como *TF-IDF*, *BM25* e a Similaridade de Cosseno (*Cosine Similarity*) baseada em representações vetoriais (*Embeddings*) são essenciais. Já na era atual, o Mecanismo de Atenção (*Attention Mechanism*) e algoritmos de decodificação estocástica (como *Top-p* e *Temperatura*) são os responsáveis por governar como a rede neural escolhe a próxima palavra a ser gerada.

Esses algoritmos são construídos com base no Machine Learning (ML), o subcampo da IA que permite aos sistemas aprenderem padrões autônoma e

estatisticamente a partir de grandes volumes de dados. O ML divide-se em paradigmas fundamentais: o Aprendizado Supervisionado, onde o modelo é treinado com entradas e respostas corretas rotuladas (muito comum na classificação de intenções clássica); o Aprendizado Não Supervisionado, que agrupa dados não rotulados por similaridade semântica para descobrir novos padrões; e o Aprendizado por Reforço, onde um agente maximiza recompensas baseadas em tentativas e erros no ambiente. No escopo deste trabalho, não houve o treinamento de um modelo a partir do zero, mas sim a inferência sobre o modelo fundacional *Qwen 2.5-7B-Instruct*. Este LLM foi originalmente construído com uma complexa combinação desses métodos, passando por aprendizado não supervisionado em bilhões de textos, seguido de ajustes supervisionados e Aprendizado por Reforço a partir de Feedback Humano (RLHF) para garantir um comportamento seguro. Em contrapartida, a lógica primária do código desenvolvido baseia-se em recuperação determinística (consulta à base JSON), utilizando técnicas clássicas de processamento léxico.

A ponte entre esses algoritmos matemáticos e a comunicação humana é estabelecida pelo Processamento de Linguagem Natural (PLN). Dentre as técnicas fundamentais aplicadas neste projeto, destaca-se a Tokenização, realizada por meio da biblioteca NLTK (*Natural Language Toolkit*), que divide as frases do usuário em unidades menores isoladas (tokens), permitindo a remoção de pontuações e a busca eficiente na base de conhecimento local. Outro pilar é a Análise Semântica, responsável por extrair o significado real por trás das palavras considerando as relações de contexto. Na etapa local, o projeto executa uma semântica simplificada por pareamento léxico; contudo, durante o *fallback*, a análise semântica profunda é assumida pelo LLM. Essa etapa generativa é suportada pelos Modelos de Linguagem baseados em Transformers, arquitetura que substituiu as antigas redes recorrentes ao analisar todas as palavras simultaneamente através do mecanismo de atenção. O uso desta tecnologia via API garante que o chatbot, ao não encontrar a resposta na base estruturada, produza retornos longos, coerentes e cientificamente embasados de forma dinâmica.

### **3. Estado da Arte**

O avanço contínuo do Processamento de Linguagem Natural (PLN) tem impulsionado a transição de chatbots baseados em regras para agentes conversacionais embasados em Grandes Modelos de Linguagem (LLMs). O estado da arte atual foca em otimizar a confiabilidade, a segurança e a eficiência computacional desses modelos em domínios específicos. Ao desenvolver o FitBot, a arquitetura híbrida adotada foi diretamente confrontada com os desafios inerentes à tecnologia atual, permitindo um paralelo direto com pesquisas recentes da literatura científica.

#### **3.1. Alucinações do Chatbot e do LLM**

Modelos generativos são projetados para prever a próxima palavra com base em probabilidades estatísticas, o que frequentemente resulta na criação de informações plausíveis, porém factualmente incorretas (alucinações). No ecossistema *fitness*, inventar a execução de um exercício ou a composição de um macronutriente pode gerar riscos reais de lesão ou prejuízo metabólico ao usuário.

Para mitigar esse problema no FitBot, adotou-se uma dupla barreira de contenção. A primeira foi o ajuste matemático: o hiperparâmetro de Temperatura (*temperature*) foi reduzido para 0.2, forçando o modelo a adotar um comportamento quase determinístico e priorizando a exatidão em detrimento da criatividade. A segunda barreira consistiu em uma rigorosa Engenharia de *Prompt*. O escopo do agente foi blindado com diretrizes imperativas, proibindo explicitamente a invenção de dados e instituindo uma mensagem de recusa padrão para tentativas de desvio de assunto (como política ou esportes automotivos). Essa abordagem inibiu a IA de tentar "adivinhar" respostas, forçando-a a admitir limitações quando necessário.

### 3.2. Dependência de Dados

A confiabilidade primária do FitBot depende diretamente da qualidade da sua base de conhecimentos (o arquivo JSON local), exigindo curadoria e manutenção constante. Quando a busca local falha e o sistema entra em modo generativo, ele passa a depender exclusivamente do conhecimento pré-treinado do modelo Qwen 2.5, o que gera limitações temporais (corte de dados do treinamento) e possíveis respostas enviesadas pela literatura majoritária da internet.

Na literatura científica recente, o estado da arte propõe abordagens complementares para o controle do modelo. O artigo "*Configurable Safety Tuning of Language Models with Synthetic Preference Data*", de Víctor Gallego (2024), busca aumentar o controle sobre o comportamento do modelo focando especificamente nas políticas de segurança e censura das respostas. Em vez de depender puramente de uma base externa de conhecimento ou de um bloqueio engessado, o autor propõe o *Configurable Safety Tuning* (CST). Esta técnica avançada permite alternar dinamicamente entre diferentes níveis de segurança e comportamento por meio de *prompts* de sistema durante o momento da inferência, oferecendo uma camada extra de controle sem depender de atualizações massivas de dados.

### 3.3. Custos Computacionais e Tempo Médio de Resposta

Enquanto a etapa de recuperação da base de dados local do FitBot responde em milissegundos, o acionamento do LLM via *Inference Router* do Hugging Face introduz uma latência considerável. Nos testes realizados, o tempo médio de resposta do modelo generativo situou-se entre 10 e 15 segundos. Esse atraso é resultado de uma soma de fatores operacionais: a latência de rede (ida e volta da

requisição HTTP), o processamento da janela de contexto (histórico da conversa acoplado ao *prompt*) e o tempo de inferência necessário para a rede neural gerar gradativamente até 400 *tokens* de saída. Em larga escala, essa latência compromete a experiência do usuário e gera altos custos de infraestrutura em APIs tarifadas.

Para solucionar esse gargalo computacional, o estado da arte aponta para técnicas como a **Decodificação Especulativa** (*Speculative Decoding*). Conforme abordado em pesquisas recentes sobre aceleração de inferência, essa técnica utiliza um modelo menor (e extremamente rápido) para "chutar" ou prever os próximos *tokens*, enquanto o modelo maior e mais pesado apenas valida essas previsões em paralelo. Essa abordagem arquitetural acelera a velocidade de geração do texto em até 5 vezes, reduzindo drasticamente o tempo de resposta sem nenhuma perda na qualidade matemática ou semântica do texto gerado.

### 3.4. Questões Éticas

Ao assumir a *persona* de um personal trainer e nutricionista sênior, o chatbot lida diretamente com a biologia e a saúde do usuário. A linha entre fornecer "orientações educacionais" e realizar uma "prescrição clínica não autorizada" é tênue. Um conselho nutricional genérico pode ser inofensivo para a maioria, mas perigoso para um indivíduo com condições clínicas pré-existentes ou transtornos alimentares. Do ponto de vista ético e legal, uma IA não possui registro profissional para prescrever tratamentos médicos. Para contornar essa questão de forma imediata, o FitBot incluiu uma instrução obrigatória de isenção de responsabilidade (*disclaimer*) no núcleo do *prompt*, determinando que a IA nunca substitua a orientação médica.

Traçando um paralelo com o estado da arte, o já citado artigo "*Configurable Safety Tuning of Language Models with Synthetic Preference Data*" resolve um dilema ético profundo da indústria de IA. Tradicionalmente, as empresas criam um alinhamento ético "de fábrica" muito rígido, o que gera falsas recusas e modelos excessivamente censurados. O método proposto por Gallego utiliza a Otimização Direta de Preferência (*Direct Preference Optimization* - DPO) baseada em dados sintéticos para permitir que as configurações éticas e de segurança sejam parametrizáveis (*sliders*) no momento do uso. Isso possibilita que aplicações como o FitBot ajustem o rigor de suas recusas de acordo com o contexto do usuário final, equilibrando segurança clínica com utilidade conversacional.

## 4. Metodologia

O desenvolvimento deste trabalho orientou-se pela construção de um assistente virtual temático, denominado **FitBot**, estruturado sob o paradigma

cliente-servidor e baseado em uma arquitetura de inteligência artificial híbrida. A construção metodológica do sistema dividiu-se em escolhas temáticas, aprimoramento de infraestrutura de interface e engenharia de *prompt* com gestão de memória conversacional.

#### 4.1. Justificativas Técnicas e Temática

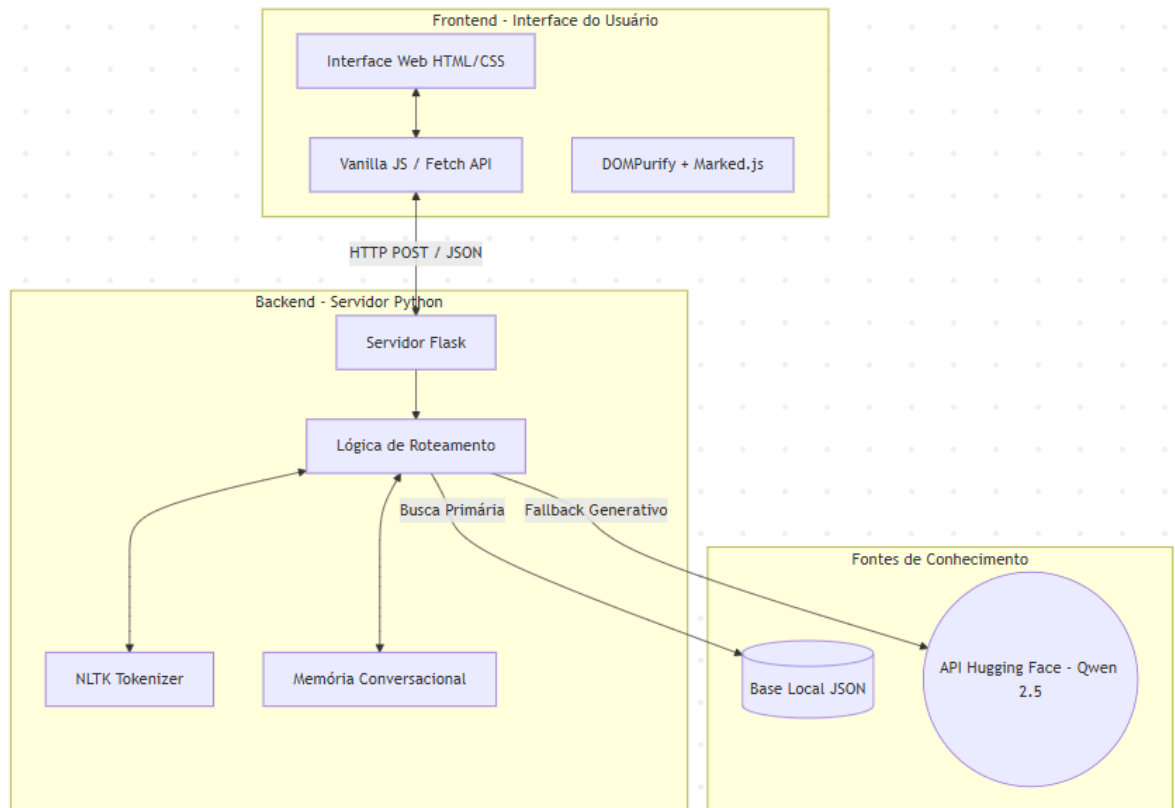
A concepção do projeto exigiu a definição de um escopo rigoroso e a seleção de tecnologias adequadas para suportar o processamento de linguagem natural e a segurança da aplicação.

- **Restrição de Domínio Temático:** A escolha do ecossistema de saúde, nutrição esportiva e musculação justifica-se pela alta densidade de informações críticas. Para evitar que o modelo respondesse a perguntas fora do escopo (como política ou esportes não correlatos), implementou-se uma diretriz estrita via Engenharia de *Prompt*, garantindo que a IA atue exclusivamente como um especialista na área;
- **A Escolha do Modelo e Ajuste de Parâmetros:** Optou-se pelo modelo *open-source* **Qwen 2.5-7B-Instruct** via *Inference Router* da Hugging Face. Para garantir respostas consistentes e baseadas em evidências, mitigando alucinações matemáticas da rede neural, o hiperparâmetro de Temperatura (*temperature*) foi ajustado para **0.2**, priorizando a exatidão determinística em detrimento da variância criativa;
- **Segurança e Renderização:** Como diferencial técnico de segurança, o projeto incorporou a proteção de chaves de API utilizando variáveis de ambiente (`python-dotenv`). No *Frontend*, implementou-se a biblioteca **DOMPurify** para higienizar o conteúdo gerado pela IA, mitigando riscos de ataques de injeção de código (XSS), operando em conjunto com a biblioteca **Marked.js** para a renderização de tabelas e formatações ricas em *Markdown*.

#### 4.2. Arquitetura do Sistema

A arquitetura do FitBot foi desenhada para separar claramente as responsabilidades de interface de usuário (UX/UI), lógica de negócios e consumo de inteligência artificial externa.

**Figura 1 - Diagrama da arquitetura do sistema ilustrando a comunicação entre o Frontend, Backend e os serviços de inteligência**



Fonte: Elaborada pelo autor (2026).

Os componentes arquiteturais operam da seguinte forma:

- **Frontend (Interface e Apresentação):** Construído com HTML5, CSS3 e *Vanilla JavaScript*. Utiliza a *Fetch API* para comunicação assíncrona. A interface possui capacidade de processar saídas complexas da IA — como tabelas de treino e listas de dietas —, convertendo *Markdown* bruto em elementos HTML estruturados e higienizados.
- **Backend (Servidor e Orquestração):** Operacionalizado pelo micro-framework **Flask** (Python 3.12). Atua como o maestro da aplicação, interceptando as requisições HTTP, executando a lógica de decisão de camadas (Base vs. LLM) e administrando o histórico de sessões.
- **Integrações Externas:** O sistema consome uma base de dados local estática (*base\_conhecimento.json*) para repostas exatas de baixa latência, e integra-se dinamicamente à API RESTful do Hugging Face para o raciocínio generativo complexo.



### 4.3. Construção do Agente e Gerenciamento de Contexto

O núcleo de processamento do agente conversacional opera em um fluxo sequencial estrito, com destaque para o mecanismo recém-implementado de memória de curto prazo:

1. **Pré-processamento (NLTK):** A mensagem recebida passa por uma filtragem léxica. Utilizando a função `word_tokenize` da biblioteca *Natural Language Toolkit* (NLTK), a frase é fragmentada em unidades atômicas (tokens), convertida para minúsculas e desprovida de pontuações, gerando um vetor de palavras-chave.
2. **Recuperação de Informação Local (Consulta Estruturada):** O algoritmo varre a base de conhecimento (JSON). O agente agrupa os tokens das chaves cadastradas e verifica sua presença no texto do usuário (ex: "supino", "creatina"). Se houver correspondência exata, o fluxo é interrompido e a resposta padronizada é devolvida imediatamente, economizando tokens e latência de rede.
3. **Mecanismo de *Fallback* e Memória Conversacional (LLM):** Caso a busca local falhe, o sistema aciona a IA generativa utilizando um gerenciamento de contexto avançado:
  - **Engenharia de Prompt Externalizada:** O servidor carrega um arquivo de instruções primárias (`fitbot_prompt.txt`), injetando o escopo de atuação e as regras de restrição de domínio na *role* de `system`.
  - **Janela Deslizante de Histórico (*Sliding Window*):** Para resolver o problema de amnésia conversacional sem estourar o limite de *tokens* do modelo, implementou-se uma lista de histórico (`self.historico`). O sistema concatena as mensagens anteriores (limitadas às últimas 10 interações via técnica de fatiamento de vetor) junto à nova pergunta do usuário.
  - **Geração:** O pacote de dados, contendo a instrução do sistema, o contexto das interações anteriores e a dúvida atual, é enviado à API. O LLM avalia a semântica, redige a resposta em *Markdown* e a devolve ao servidor, que atualiza a memória da sessão e entrega a orientação final ao cliente.

## 5. Resultados e Discussão

A implementação da arquitetura híbrida no FitBot demonstrou resultados práticos altamente satisfatórios, validando a premissa de que a união entre recuperação determinística (Base de Conhecimento) e geração contextual (LLM)

eleva significativamente a qualidade da interação humano-computador em domínios específicos.

### 5.1. Avaliação de Desempenho e Uso da LLM

A integração com o modelo **Qwen 2.5-7B-Instruct** via *Inference Router* apresentou uma fluidez conversacional notável. A transição entre as respostas obtidas instantaneamente pela varredura do arquivo JSON (Base Local) e as respostas geradas dinamicamente pelo LLM no mecanismo de *fallback* ocorreu de maneira transparente, mantendo um tempo médio de resposta aceitável para aplicações *web*.

A implementação da memória de curto prazo (técnica de *sliding window* com as 10 últimas interações) resolveu o problema inicial de "amnésia" do modelo. Durante os testes, o agente demonstrou capacidade de manter o raciocínio ao longo de múltiplos turnos de conversa, recordando-se de objetivos específicos do usuário (como o desejo de focar em um músculo particular ou restrições alimentares) sem a necessidade de o usuário repetir o contexto a cada nova mensagem.

### 5.2. Qualidade e Abrangência das Respostas

A avaliação qualitativa das respostas geradas pela IA revelou um profundo domínio sobre uma vasta gama de assuntos do universo *fitness* e *bodybuilding*. O modelo foi capaz de transitar com segurança entre explicações sobre biomecânica de exercícios, fisiologia do metabolismo, organização de dietas e até mesmo a história do fisiculturismo.

Um dos grandes destaques do resultado final foi a capacidade de o sistema estruturar o conhecimento de forma visualmente acessível. Graças ao ajuste de *prompt* aliado à renderização no *Frontend* (utilizando as bibliotecas *Marked.js* e *DOMPurify*), a IA foi capaz de gerar rotinas de treinos e tabelas de macronutrientes perfeitamente formatadas em *Markdown*. O uso contínuo de listas, tópicos e tabelas fragmentou a complexidade das informações, tornando a leitura muito mais intuitiva para o usuário final.

### 5.3. Controle de Domínio e Aderência às Regras

O sucesso da Engenharia de *Prompt* foi comprovado pela estrita obediência do modelo às regras de sistema impostas. Destacam-se dois resultados cruciais nesta camada de controle:

- **Respeito aos Limites Formais:** O ajuste do hiperparâmetro de Temperatura para **0.2** reduziu drasticamente as alucinações. Além disso, o chatbot respeitou o limite máximo de palavras, entregando respostas concisas e diretas sem cortar o raciocínio no meio de uma frase (problema mitigado pela regra atribuída no *prompt* do chatbot "*Limite todas as respostas a no máximo 300 palavras.*").

- **Restrição de Escopo (Efeito "Guarda-Costas"):** Durante os testes de estresse, quando o modelo foi intencionalmente provocado a responder sobre temas alheios ao seu treinamento (como Fórmula 1, política ou finanças), o agente acionou corretamente sua trava de segurança. Ele declinou o atendimento de forma educada, respondendo exatamente a frase programada: *"Sou especializado apenas em treino, nutrição esportiva, alimentação saudável e condicionamento físico. Não posso ajudar com esse assunto."*

## 6. Conclusões e Trabalhos Futuros

O desenvolvimento do FitBot confirmou a eficácia da aplicação de inteligência artificial generativa aliada a técnicas clássicas de processamento de linguagem natural no cenário de suporte especializado. O projeto demonstrou que a arquitetura híbrida — que intercala uma base de conhecimento local e um Grande Modelo de Linguagem (LLM) — é a abordagem ideal para sistemas que exigem, simultaneamente, precisão técnica e fluidez conversacional.

O principal aprendizado extraído desta implementação diz respeito ao **controle de respostas**. Ficou evidente que um LLM puro, por mais avançado que seja, tende a dispersar ou alucinar quando submetido a perguntas fora de seu escopo ideal. A solução arquitetural adotada provou ser altamente eficaz: o uso do *NLTK* garantiu o bloqueio determinístico e o acesso rápido a dados críticos locais, enquanto a Engenharia de *Prompt* (aliada ao ajuste de temperatura para 0.2 e ao limite de tokens) atuou como uma barreira semântica estrita, mantendo o Qwen 2.5 restrito ao domínio de saúde e musculação. Além disso, a estratégia de memória conversacional baseada em janela deslizante (*sliding window*) solucionou o problema da perda de contexto sem onerar o consumo da API.

Entretanto, o projeto revelou limitações inerentes à **dependência do modelo**. Ao delegar o raciocínio complexo à infraestrutura do Hugging Face via *Inference Router*, o sistema tornou-se dependente da estabilidade da conexão de rede e da disponibilidade do servidor de terceiros. Durante o desenvolvimento, oscilações de latência evidenciaram que arquiteturas baseadas inteiramente em APIs externas em nuvem podem sofrer com gargalos de tempo de resposta (*timeouts*).

Diante das conclusões obtidas, elencam-se as seguintes propostas para **Trabalhos Futuros**:

1. **Inferência Local e Quantização:** Substituir a chamada de API em nuvem pela execução de um modelo LLM quantizado localmente (utilizando ferramentas como *Ollama* ou *Llama.cpp*), eliminando a dependência de internet e garantindo maior privacidade dos dados do usuário;
2. **Evolução para um Sistema RAG Completo:** Substituir a busca estática no arquivo JSON local por um Banco de Dados Vetorial (como *ChromaDB* ou

*Pinecone*). Isso permitiria ao sistema realizar buscas por similaridade semântica em milhares de artigos científicos de nutrição esportiva, não dependendo apenas de correspondências exatas de palavras-chave;

3. **Autenticação e Personalização:** Desenvolver um sistema de banco de dados relacional (ex: PostgreSQL) para criar perfis de usuários autênticos, permitindo que a IA acompanhe o progresso de cargas, o percentual de gordura e o histórico de evolução do usuário a longo prazo.

## 7. Referências

**BIRD, S.; KLEIN, E.; LOPER, E.** Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit. 1. ed. Pequim: O'Reilly Media, 2009.

**QWEN TEAM.** Qwen2.5: A Party of Foundation Models. Repositório e Documentação Técnica. Hugging Face, 2024. Disponível em: <https://huggingface.co/Qwen>.

**SIRQUEIRA, T.** Inteligência Artificial e Machine Learning. Material de Aula. Juiz de Fora: UniAcademia Centro Universitário, 2026.

## 8. Apêndices

a) Códigos Fonte: <https://github.com/Igor0155/ChatBot-Academia>

b) Vídeo: <https://youtu.be/rRR8z48A26E>